

RFC · HARBOR SATELLITE · DRAFT FOR DISCUSSION

Trusted-Peer Artifact Distribution for Harbor Satellite — Air-Gapped P2P Image Proxying with Digest-Graph Integrity

Upstream: harbor-satellite#542

Program: cncf/mentoring#1999

Author: Harshita Kumari · Harshitaakri

Updated 2026-08-18

Status	Draft for discussion
Primary Author	Harshita Kumari (Harshitaakri)
Collaborators	—
Reviewers	@NucleoFusion (Lakshit Singh), @bupd (Prasanth Baskar), @Vad1mo (Vadim Bauer), @OrlinVasilev (Orlin Vasilev) — Harbor Satellite maintainers/mentors
Upstream tracking	harbor-satellite#542 · program listing cncf/mentoring#1999 · digest-drift fix PR #637 · ADR-0010 draft PR #638
Companion experiment	oci-closure-verify — Go verification harness against real local OCI registries: root-visible-last publication, fail-closed missing-manifest handling, digest-verified corruption detection, singleflight dedup (recorded demo)
Last updated	2026-08-18

Approvers

Name	yes	not yet	Date
@NucleoFusion			
@bupd			
@Vad1mo			
@OrlinVasilev			

Table of Contents

1. [Overview](#)

2. [Motivation](#)

3. [Requirements](#)

4. [Background — the current architecture, verified at code level](#)

5. [Goals and Non-Goals](#)
6. [Product & User Use Cases](#)
7. [Gap Analysis — what is and isn't captured upstream](#)
8. [Prior Art & Landscape](#)
9. [Design](#)
10. [Reference Experiment — reproducible correctness demonstration](#)
11. [Failure-Mode Matrix](#)
12. [Performance & Resource Envelope](#)
13. [Timeline](#)
14. [Dependencies](#)
15. [Alternatives Considered](#)
16. [Operations](#)
17. [Security / Privacy / Compliance](#)
18. [Risks](#)
19. [Future Work](#)
20. [Open Questions for Maintainers](#)
21. [Audit Log](#)
22. [References](#)

Overview

Harbor Satellite makes OCI images and artifacts available at edge locations. When a Satellite in an air-gapped or intermittently connected deployment misses an artifact, its only recovery paths today are a central registry, Ground Control, or manually imported offline media — even when **another trusted Satellite on the same local network already holds the exact content**.

This RFC proposes an **opt-in trusted-peer distribution mode**: a Satellite consults a configured set of trusted local peers, determines which peer holds a requested OCI artifact, and copies that artifact into its own local content store — with **no Internet and no Ground Control connectivity required**, and with OCI digest-based integrity preserved end-to-end.

The design rests on one structural observation and one correctness principle:

- **Structural**: every Satellite already fronts an OCI Distribution API — today via embedded Zot, in the target architecture via the transparent proxy serving retained content (ADR-0009). Therefore **the peer read path requires no new wire protocol**. The engineering surface is *peer selection*, *trust*, and *transfer correctness* — not transport invention.
- **Correctness**: an OCI artifact is a **descriptor graph**, not a manifest. "Peer has the manifest" is not "peer has the artifact," and "we fetched the manifest" is not "the artifact may become visible." The transfer engine therefore enforces closure-complete, digest-verified, **root-last** publication with idempotent crash recovery (§9.3–§9.4), so that a partially transferred artifact can never be observed by a consumer.

The proposal is deliberately shaped to survive the ADR-0009 transition (Crane → ORAS, Zot leaving the target architecture): peers are modeled as one more **content source** behind the same resolve/fetch/copy abstraction that ADR-0009 mandates, selected after local content and before (or instead of) upstream.

Motivation

Edge fleets exist precisely where connectivity is worst: retail branches, factory floors, vessels, defense sites. Operational reality in such fleets:

1. **Rollout waves are LAN-redundant.** When a desired-state update adds `app:v42` to 30 Satellites on one site network, today each Satellite independently pulls the same bytes across the constrained uplink — or waits for media. The first Satellite to hold the artifact could serve the other 29 at LAN speed.
2. **Air-gapped sites bootstrap by sneakernet.** Content is imported to *one* Satellite via offline media; propagating it to N others currently means N manual imports. Trusted-peer distribution turns one import into a site-wide seed.
3. **Ground Control outages should degrade, not disable.** A Satellite that misses content during a GC outage has a healthy neighbor with the bytes; the current architecture cannot use it.
4. **Bandwidth cost is asymmetric.** Uplink (satellite/cellular/VPN) is expensive and slow; LAN is abundant. Every peer-served byte is an uplink byte saved.

The upstream issue (#542) and program listing name exactly this capability, and require the design to "work without Internet or Ground Control connectivity and preserve OCI digest-based integrity verification."

Requirements

Functional requirements

ID	Requirement	Source
FR1	Opt-in configuration of trusted peer endpoints, timeouts, retry policy, concurrency limits, and feature enablement; zero behavior change when disabled	Listing expected outcome
FR2	Peer selection and artifact-availability checks for OCI manifests and blobs , addressed by digest	Listing expected outcome
FR3	A peer-to-peer transfer path that imports requested artifacts into the local Satellite content store and verifies manifest and blob digests	Listing expected outcome
FR4	Deterministic fallback behavior when no eligible peer is reachable (upstream if connected; explicit failure otherwise)	Listing expected outcome
FR5	Metrics and structured logs for peer attempts, transfer outcomes, bytes transferred, and cache-fill results	Listing expected outcome
FR6	Unit and integration tests, including a multi-Satellite air-gapped test environment demonstrating successful transfer and failure recovery	Listing expected outcome
FR7	Works for container images and arbitrary OCI artifacts (Helm charts, SBOMs, signatures, Wasm, models) — the content model is descriptor graphs	ADR-0009 alignment
FR8	Operator documentation: configuration and troubleshooting	Listing expected outcome

Non-functional requirements

ID	Requirement
NFR1	No Ground Control or Internet dependency on the peer path; all decisions computable from local desired state + peer responses
NFR2	Digest-preserving transfer: bytes stored locally hash to exactly the digests advertised by the descriptor graph (no media-type rewriting on the peer path; see §9.4)
NFR3	BYO-registry compatible : the baseline uses only OCI Distribution primitives, so it works whether local storage is embedded Zot today, a bring-your-own registry (docker-compose.byo.yml), or the ADR-0009 ORAS OCI layout tomorrow
NFR4	Edge resource envelope : streaming I/O (no whole-blob buffering), bounded concurrency, optional bandwidth cap
NFR5	Fail-closed : any verification failure results in no visible artifact and a demoted peer, never a partially visible artifact
NFR6	Survives the ADR-0009 architecture transition without redesign (source abstraction, §9.1)

Background — the current architecture, verified at code level

The following facts were verified directly against [main](#) (clone of 2026-08-17) and anchor every design decision below.

#	Verified fact	Where	Implication for this design
B1	Replication is state-driven: Ground Control desired state → <code>Replicator</code> interface → local registry	<code>internal/satellite/state/replicator.go</code> , <code>state_process.go</code>	The peer path slots in as an alternative <i>source</i> for satisfying desired state, not a new subsystem
B2	<code>BasicReplicator</code> builds tag-based references (<code>Entity{Name, Repository, Tag, Digest}</code>), refs formatted as <code>repo/name:tag</code>) and re-resolves at copy time	<code>replicator.go</code> (<code>Replicate</code>)	Confirms the community-reported tag/digest divergence risk at the struct level; the peer path must be digest-addressed end-to-end , with tags resolved once at the desired-state source
B3	The current copy path is single-image oriented (<code>desc.Image()</code>) and rewrites media types (<code>mutate.MediaType(img, types.OCIManifestSchema1)</code>), which changes the stored digest relative to the source	<code>replicator.go</code>	Two consequences: multi-platform indexes are not copied as indexes, and locally stored digests can differ from upstream digests. §9.4 defines the digest-domain rule the peer path must follow
B4	The replicator already checks destination blob existence before pulling, "saving bandwidth on crash recovery"	<code>replicator.go</code> doc comment	Content-addressed idempotency

#	Verified fact	Where	Implication for this design
			is an accepted pattern in this codebase; §9.3's resume semantics extend it to graph closure
B5	ADR-0009 (proposed, 2026-08-03) pivots Satellite to a policy-enforcing transparent OCI proxy: <code>oras-go</code> replaces Crane for resolution, graph transfer, metadata, and replication; "Zot and bbolt are not part of the target architecture" ; local content moves to an ORAS OCI image layout; the proxy pipeline is gate → evidence → execute with fail-closed policy	<code>docs/decisions/0009-transparent-oci-registry-proxy.md</code>	The peer feature must not couple to Zot. Modeled as a content <i>source</i> behind ADR-0009's "one resolve, fetch, copy, and metadata model," it is valid on both sides of the transition
B6	<code>oras.land/oras-go/v2 v2.6.0</code> is currently an indirect dependency; the ORAS rework is not yet implemented	<code>go.mod</code>	There is a live co-design window : the peer source can be built ORAS-native rather than retrofitted
B7	Embedded Zot <code>v2.1.16</code> today (<code>internal/satellite/registry/</code>), and a BYO-registry deployment mode exists	<code>go.mod</code> , <code>docker-compose.byo.yml</code>	Baseline peer serving must assume nothing beyond OCI Distribution API (NFR3)
B8	ADR-0005 (accepted) : SPIFFE/SPIRE identity for Satellites; mutual authentication; Zero-Touch Registration; TLS config carries <code>cert_file</code> / <code>key_file</code> / <code>ca_file</code> / <code>skip_verify</code>	<code>docs/decisions/0005-spiFFE-identity-and-security.md</code> , <code>pkg/config/config.go</code>	The trust plane for peer↔peer mTLS already exists ; this RFC extends it rather than

#	Verified fact	Where	Implication for this design
			inventing identity (§9.6)
B9	ADR-0006 (accepted) : Satellite lifecycle states <code>pending / online / stale / revoked</code> , SPIFFE-conventioned	<code>docs/decisions/0006-satellite-lifecycle-states.md</code>	Peer eligibility can consume lifecycle state where available: <code>revoked</code> ⇒ never a peer
B10	Structured audit logging with syslog and OTel sinks is already configurable	<code>pkg/config/config.go</code> , <code>docs/guides/audit-logging.md</code>	Peer imports emit audit events through the existing pipeline; no new telemetry stack (§9.8)
B11	An e2e harness exists (<code>test/e2e/{docker, testconfig, testdata}</code> , Taskfile-driven compose)	<code>test/e2e/</code>	The multi-Satellite air-gapped environment (FR6) extends this harness instead of introducing a new one

Goals and Non-Goals

Goals

- G1 — An **approved ADR** covering architecture, trust model, failure modes, and headless/air-gapped compatibility. ADR-0010 is drafted upstream as PR #638; this document is the extended design record backing it.
- G2 — Opt-in trusted-peer configuration model (FR1) with strict validation and fail-closed semantics.
- G3 — Digest-addressed availability checking and **closure-complete, digest-verified, root-last** transfer (FR2–FR3, §9.3–§9.4).
- G4 — Deterministic fallback chain and peer health management (FR4, §9.5).
- G5 — First-class observability: metrics, structured logs, audit events (FR5, §9.8).

- G6 — Multi-Satellite air-gapped integration environment with fault injection, seeded by the reference experiment (FR6, §10).
- G7 — Operator documentation (FR8).

Non-Goals (v1)

- N1 — **Dynamic peer discovery** (mDNS/DNS-SD/DHT/gossip). v1 peers are explicitly configured; discovery is future work layered on the same source abstraction (§19).
- N2 — **Untrusted or semi-trusted peers**. v1 trust is binary and allowlisted.
- N3 — **WAN/cross-site federation**. Scope is the local network segment.
- N4 — **Changes to signing/verification trust roots**. Integrity continues to come from OCI digests; signature verification remains whatever policy the Satellite already enforces.
- N5 — **Replacing desired-state replication**. The peer path is a *source optimization* for satisfying desired state (and, in the proxy model, cache fill) — not a new state protocol.
- N6 — **Peer-to-peer push**. v1 is pull-only from the requesting Satellite's perspective; nothing is written to a peer.

Product & User Use Cases

#	Scenario	Without peers	With trusted-peer distribution
U1	Rollout wave on one site — desired state adds <code>app:v42</code> to 30 Satellites behind one uplink	30 uplink pulls of identical bytes	1 uplink pull; 29 LAN fills; uplink bytes reduced ~29/30 for that artifact
U2	Fully air-gapped site — content arrives on offline media at one Satellite	N manual imports	One import; peers fill from the seeded Satellite with digest verification
U3	Ground Control outage — a Satellite missed a wave and GC is unreachable	Stuck until GC returns	Fills from any online trusted peer holding the closure
U4	Constrained uplink (vessel/cell)	Every miss pays uplink latency/cost	Availability check is O(HEAD)s on LAN; transfer at LAN speed
U5	BYO-registry deployment	—	Same behavior: baseline requires only Distribution primitives (B7)
U6	Arbitrary OCI artifacts — Helm chart with provenance, SBOM-attached image	Same gaps	Same path: closure walker treats every artifact as a descriptor graph (FR7)

Gap Analysis — what is and isn't captured upstream

Captured in #542 and community discussion

The upstream thread already contains valuable ground-truth that this design treats as **constraints**, credited to the contributors who established it:

- **Tag/digest divergence is real and reproducible** — a mutable tag moving between desired-state recording and copy time causes the destination to receive different bytes than recorded (community reproduction by [1amKhush](#)). B2/B3 confirm the mechanism at code level. Constraint absorbed: peer protocol is digest-only; peers are never asked about tags (§9.2).
- **The architecture is mid-transition** — ADR-0009 supersedes the Crane/Zot target state, and documented Zot sync behavior was found to be gated behind build tags absent from normal builds (community archaeology by [shreyashsri79](#)). Constraint absorbed: no coupling to Zot or its sync extension; source abstraction survives the transition (§9.1).
- **BYO-registry portability matters** — raised by [harshal0704](#) , whose adjacent state-semantics work (deletion states, re-added artifacts) also informs the local-store interaction. Constraint absorbed as NFR3.

What this RFC adds on top of the prior discussion

1. **The digest-domain problem (B3)**. Today's copy path rewrites manifest media types, so local digests can differ from source digests. No prior discussion defines which digest domain the peer path verifies against. §9.4 does, and makes the peer path digest-preserving. The source-domain half is already being corrected by the author's PR #637, which makes replication prefer the recorded digest when available — closing the moved-tag window — with a regression test covering the case.
2. **Closure-complete, root-last publication as the atomicity mechanism**, with a precise definition of "closure" (including how referrers are and aren't part of it) and of "addressable" (§9.3–§9.4).
3. **Idempotent crash recovery and per-digest singleflight**, extending the codebase's existing blob-presence pattern (B4) to graph scope.
4. **A trust model that extends accepted ADR-0005 SPIFFE identity** instead of inventing peer identity, with an explicit threat table (§9.6).
5. **A 10-row failure-mode matrix** with detection, behavior, and the observable signal for each (§11).
6. **A reproducible fault-injection experiment** demonstrating the violation and the invariant on real registries, offered as the seed of the FR6 integration suite (§10).
7. **A complete configuration and observability specification** consistent with existing [pkg/config](#) patterns and the audit pipeline (§9.7–§9.8).

Prior Art & Landscape

System	Model	Why it doesn't fit Satellite's constraints
Dragonfly	P2P file/image distribution with scheduler + seed peers	Requires scheduler/manager infrastructure and general connectivity; node-cache semantics rather than durable registry import; heavy for constrained edge
Uber Kraken	Tracker-based P2P registry distribution	Origin/tracker components assume datacenter topology; not designed for allowlisted, air-gapped, GC-less operation
Spegel	Kubernetes node-local OCI mirror with libp2p discovery	Cluster-internal ephemeral cache, discovery-based trust, assumes k8s networking; Satellite needs durable import into a standalone registry with statically configured trust
Zot sync extension	Registry-to-registry pull mirroring	Couples the feature to Zot, which is leaving the target architecture (B5); periodic/tag-oriented rather than digest/demand-driven; behavior gated by build tags in practice
IPFS/DHT approaches	Content-addressed swarm	Open-world discovery and trust model is the opposite of an allowlisted air-gapped fleet; operationally heavy
Cron <code>skopeo</code> / <code>crane</code> copies	Scripted mirroring	No availability logic, no atomic publication, no health/fallback, no observability; exactly the manual toil this feature removes

The niche Satellite occupies — **statically trusted peers, air-gap-first, durable digest-verified registry import, tiny footprint** — is not served by any of the above. The closest conceptual relative is Spegel's "peers already speak Distribution" insight, which this design adopts while replacing discovery-based trust with configured trust.

Design

9.1 Placement — a peer is a content source (ADR-0009-native)

ADR-0009 mandates "one resolve, fetch, copy, and metadata model" for local and remote content, with `oras-go` as the graph-transfer substrate and policy evaluated fail-closed before content is served (B5). This design adds **one new implementation of the existing concept "where content comes from"**:

```

ContentSource (concept; ORAS Target/ReadOnlyStorage in target arch)
├─ LocalStore      ─ Zot today; ORAS OCI layout in target architecture
├─ TrustedPeer     ─ this RFC: a configured peer's Distribution endpoint
├─ Upstream        ─ Harbor / configured upstream registries

```

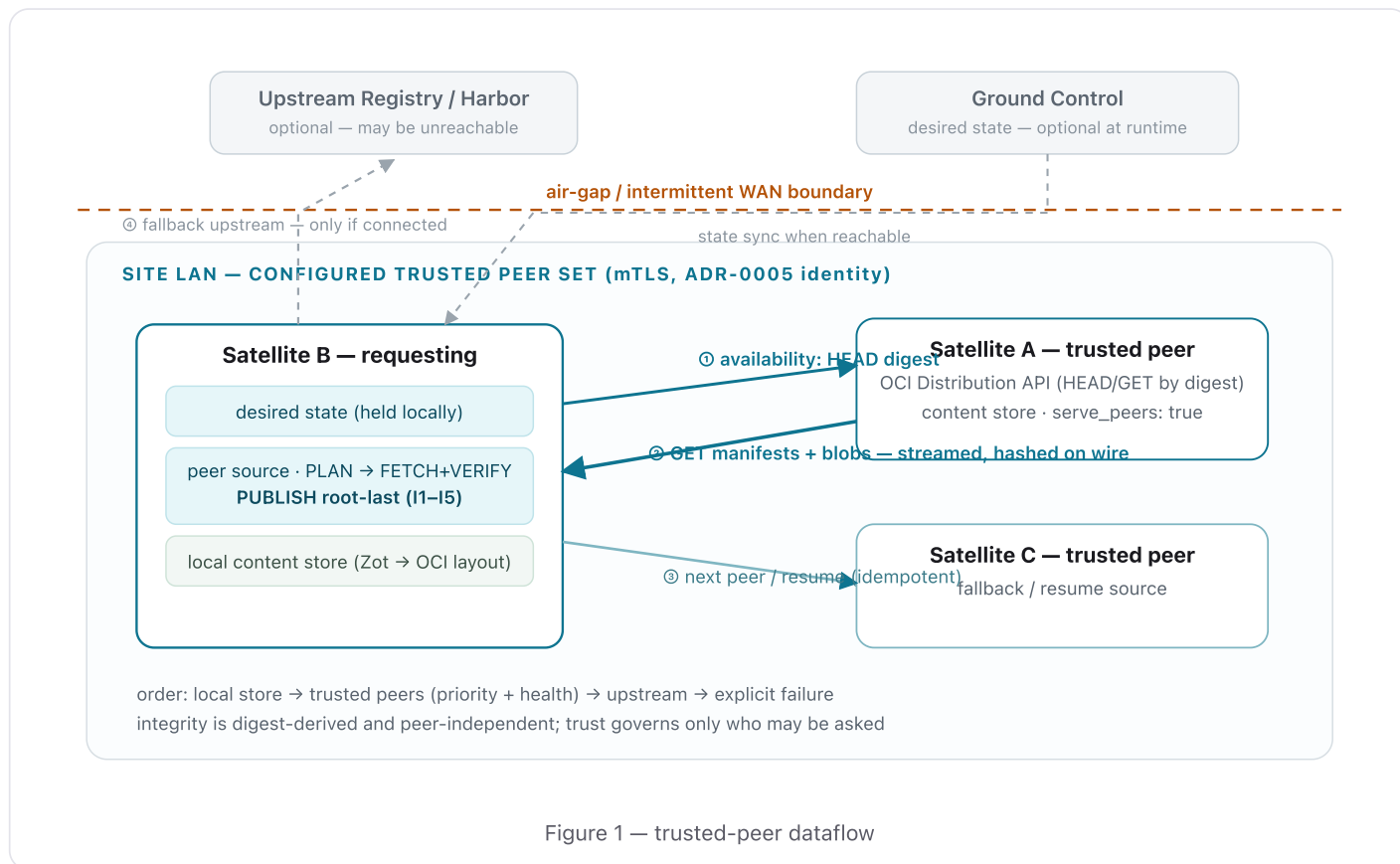
Resolution order for a required digest: **local → trusted peers → upstream (if reachable) → explicit failure**. In the target proxy pipeline, the peer source participates in stage 3 ("allowed execution: selected source") and its fetched content flows through the same verification and policy gate as upstream content — peer-sourced bytes get **no policy shortcut** (see OQ6).

Two integration modes, both behind the same source:

- **Reconcile-time fill (v1 default):** the state process, on a missing digest, consults peers before upstream. Smallest change; works headless.
- **On-demand fill (proxy mode):** a client pull that misses locally triggers a peer-first fill. Natural in the ADR-0009 proxy; enabled second.

Because `oras-go` is not yet wired (B6), v1 ships as an adapter implementing today's `Replicator`-adjacent seam while presenting the ORAS `Target`-shaped interface internally — so the same engine drops into the proxy architecture unmodified.

Figure 1 — trusted-peer dataflow across the air-gap boundary.



9.2 Peer wire protocol — the OCI Distribution API, digest-only

Every Satellite already serves Distribution (B7); the peer read path is therefore plain, spec-compliant, and BYO-portable:

Operation	Call	Notes
Manifest availability	<code>HEAD /v2/<repo>/manifests/<digest></code>	Digest reference only — peers are never asked about tags (constraint from Gap Analysis)
Blob availability	<code>HEAD /v2/<repo>/blobs/<digest></code>	Used during planning and resume
Manifest fetch	<code>GET /v2/<repo>/manifests/<digest></code>	<code>Accept</code> covers image + index + artifact media types
Blob fetch	<code>GET /v2/<repo>/blobs/<digest></code>	Streamed; hashed on the wire

No custom endpoint is required for v1. A **batched availability check** (one request answering "which of these N digests do you hold?") is a candidate optimization once N×HEAD becomes measurable overhead — deliberately deferred and listed as OQ4, since it is the only place a Satellite-specific API would leak into the peer protocol.

9.3 Transfer engine — closure, verification, and the five invariants

Definitions.

- The **closure** of a root descriptor is the set of descriptors reachable via manifest references: for an index, its child manifests and *their* closures; for a manifest, its config and layer descriptors. Every artifact — image, chart, SBOM, model — is such a graph (FR7).
- Referrers** (signatures/attestations attached via `subject`) are discovered through the referrers API, not reachable *from* the root, and are therefore **not part of the default closure**; an opt-in `include_referrers` expansion transfers them alongside (§19).
- An artifact is **addressable** when a consumer can obtain its root manifest from the local store by tag or root digest.

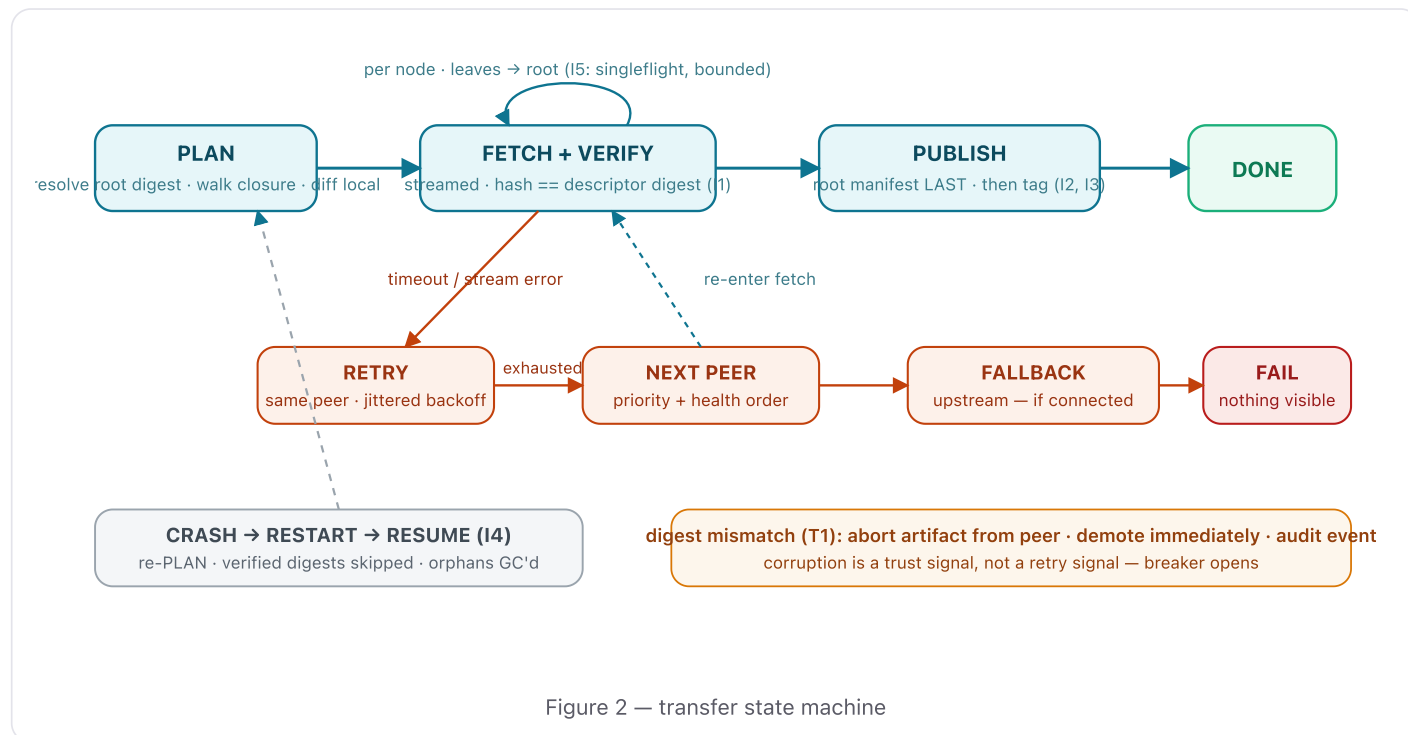
Invariants.

ID	Invariant
I1	Digest-preserving transfer. Every fetched byte stream is hashed on the wire and must equal the descriptor's digest and size; content is stored under exactly that digest. The peer path performs no media-type rewriting (contrast B3).
I2	Closure completeness before publication. All descriptors in the root's closure are present and verified locally before the artifact may become addressable.
I3	Root-last publication. Order: blobs → child manifests → root manifest/index last , then tag association. Children stored earlier are individually valid, digest-addressed content — never a broken <i>artifact</i> .
I4	Idempotent crash recovery. Presence of a verified digest is the only resume state needed; a restarted transfer re-plans the closure, skips verified content (extending B4 to graph scope), and continues. Orphaned partial content is bounded by a janitor/GC pass.
I5	Per-digest singleflight, bounded concurrency. Concurrent demands for one digest coalesce into one transfer; blob-level and transfer-level concurrency caps plus an optional bandwidth limiter honor the edge envelope (NFR4).

Engine phases: `PLAN` (resolve root digest from desired state; walk the graph — fetching only manifests — and diff against local presence) → `FETCH+VERIFY` (leaves first, streamed, hashed) → `PUBLISH` (root last, tag last) → `DONE` ;

with **RETRY** (same peer, jittered backoff) → **NEXT PEER** → **FALLBACK** (upstream) → **FAIL** (nothing visible) on error paths, and **RESUME** after crash.

Figure 2 — transfer-engine state machine with failure and recovery edges.



9.4 Publication semantics and the digest-domain rule

B3 exposes a subtlety no prior discussion has addressed: the current replicator's media-type normalization means **locally stored digests can differ from upstream digests**. If Satellite A imported an artifact through that path, the digest recorded in desired state (source domain) and the digest A actually serves (local domain) may disagree — and a peer transfer that verifies against the wrong domain will either fail spuriously or, worse, "verify" the wrong expectation.

Rule proposed by this RFC: the peer path operates entirely in the **descriptor domain of the root being transferred** — the root digest requested is the root digest published, and every child is verified against the digest embedded in its parent. The peer path never rewrites, never re-normalizes, and never consults tags. Where desired state carries a source-domain digest that local history has rewritten, the mismatch is *detected and surfaced* (metric + audit event), not papered over. Longer-term, ADR-0009's ORAS copy is itself digest-preserving, which converges the domains; the peer path is simply strict from day one. A first corrective step in the source domain already exists: the author's PR #637 makes replication use the recorded image digest when present, so a moved source tag can no longer cause the wrong image to be replicated. The remaining fleet-wide interaction is raised explicitly as OQ2 because it touches replication behavior beyond this feature.

Figure 3 — descriptor-graph closure and root-last publication order.

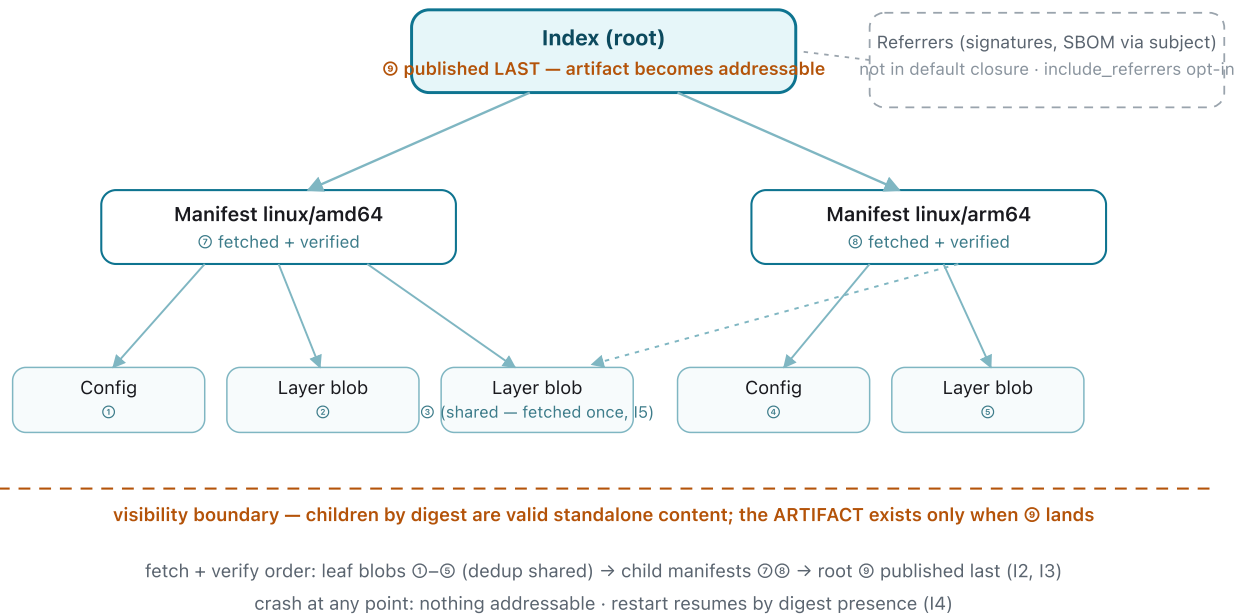


Figure 3 — closure and publication order

9.5 Peer selection, health, and fallback

- **Eligibility:** configured allowlist (§9.7); a peer marked `revoked` by lifecycle state (B9), failing TLS verification, or tripping its circuit breaker is ineligible.
- **Selection:** filter by availability (HEAD of root digest), then order by static priority, then by a health score (EWMA of recent latency/success). v1 keeps scoring simple and observable; no consensus, no gossip.
- **Circuit breaking:** consecutive failures open the breaker for a backoff window (jittered, capped); verification failures (I1) demote immediately and emit a distinct audit event — corrupt bytes are a trust signal, not a retry signal.
- **Fallback chain:** peers (in order) → upstream if configured and reachable → **explicit failure** with a machine-readable reason (`no_eligible_peer` , `closure_incomplete_on_peers` , `verification_failed` , `upstream_unreachable`). Silent hangs are treated as bugs.

9.6 Trust model

Trust answers **availability and reachability** questions ("whom may I ask; who may ask me"); **integrity comes from digests** (I1) and is peer-independent. This separation is what makes trusted-peer distribution safe in air-gapped fleets: a compromised peer can refuse service or serve garbage, but garbage cannot verify.

- **Transport & identity:** mTLS between Satellites using the existing TLS configuration surface (`cert_file` / `key_file` / `ca_file` — B8). Deployments already adopting ADR-0005 get peer identity from SPIFFE/SPIRE SVIDs with no new machinery; a pre-provisioned CA bundle covers air-gapped sites that haven't. Peer identity must match the configured peer entry (no wildcard trust).
- **Authorization (v1):** binary — a trusted peer may fetch any repository this Satellite serves, optionally narrowed by a repository allowlist per peer. Fine-grained per-artifact ACLs are future work.
- **Serving role is opt-in** separately from fetching (`serve_peers`), so operators can deploy read-only leechers.

Threat table.

#	Threat	Mitigation
T1	Peer serves wrong/corrupted bytes	I1 verification; immediate demotion + audit event; nothing published
T2	Truncated transfer (peer dies mid-blob)	Size+digest check fails; retry/next-peer; root never published (I3)
T3	Peer lies about tags	Impossible by construction — tags are never requested (§9.2)
T4	Stale peer (holds old content)	Availability is digest-presence only; "stale" content is simply <i>absent</i> for the requested digest
T5	Rogue device poses as peer	mTLS + allowlist + identity match (B8); revoked lifecycle excluded (B9)
T6	Trusted-but-compromised peer used for exfiltration via fetches	Per-peer repository allowlist; audit trail of every served digest; serving opt-in
T7	Resource exhaustion (slow-loris peer, oversized closure)	Timeouts at every phase; size ceilings from descriptor metadata; concurrency caps; breaker
T8	Replay/duplication	Content-addressed and idempotent — replay is a no-op

9.7 Configuration model

Shaped after existing **pkg/config** structures and validated fail-closed (bad peer entry ⇒ feature disabled with a logged reason, not partially enabled):

```

"peer_distribution": {
  "enabled": false,
  "serve_peers": true,
  "peers": [
    {
      "name": "sat-b",
      "url": "https://10.20.0.12:8585",
      "priority": 10,
      "repositories": ["*"],
      "tls": { "ca_file": "/etc/satellite/peers/ca.pem",
               "cert_file": "/etc/satellite/tls/client.pem",
               "key_file": "/etc/satellite/tls/client.key" }
    }
  ],
  "availability_timeout_ms": 800,
  "transfer_timeout_s": 300,
  "retry": { "max_attempts": 3, "backoff_initial_ms": 200,
             "backoff_max_ms": 5000, "jitter": true },
  "concurrency": { "blobs_per_transfer": 4, "transfers": 2 },
  "bandwidth_limit_mbps": 0,
  "include_referrers": false
}

```

Configuration arrives like all Satellite config (locally or GC-pushed) but is **fully functional with GC absent** (NFR1).

9.8 Observability

Metrics (names to be aligned with existing conventions — OQ5):

Metric	Type	Labels
<code>satellite_peer_attempts_total</code>	counter	<code>peer</code> , <code>phase</code> (availability/fetch), <code>outcome</code>
<code>satellite_peer_transfer_bytes_total</code>	counter	<code>peer</code> , <code>direction</code> (fetched/served)
<code>satellite_peer_cache_fill_total</code>	counter	<code>result</code> (peer/upstream/failed)
<code>satellite_peer_transfer_duration_seconds</code>	histogram	<code>source</code>
<code>satellite_peer_fallback_total</code>	counter	<code>reason</code>
<code>satellite_peer_health</code>	gauge	<code>peer</code>

Structured logs carry `event`, `digest`, `repository`, `peer`, `attempt`, `bytes`, `duration_ms`, `outcome`, and a per-transfer correlation ID. Peer imports and verification failures additionally emit **audit events** through the existing syslog/OTel audit pipeline (B10) — an air-gapped compliance audience cares precisely about "what entered this registry, from where, verified how."

Reference Experiment — reproducible correctness demonstration

The companion harness `oci-closure-verify` demonstrates both the violation and the invariants on real registries — no cloud, no cluster, three local containers — with a recorded run. Its four verified behaviors map one-to-one onto §9.3: root-visible-last publication → I3; fail-closed handling of missing manifests → I2 / matrix row 10; digest-verified corruption detection → I1 / T1; singleflight deduplication of concurrent requests → I5. The scenario set:

- 1. **Setup:** `peerA`, `peerC`, `local` registries; a multi-platform artifact (index → 2 platform manifests → configs + layers, 8-node closure) seeded to `peerA` and `peerC` with `crane`; full closure digests recorded.
- 2. **Violation (naive copy):** delete one platform child manifest from `peerA`; run a tag-first naive copy into `local`, publishing the index up front. Result: `crane pull --platform linux/amd64` succeeds while `linux/arm64` 404s — **a visibly broken artifact**, exactly what FR3/NFR5 forbid.
- 3. **Invariant (engine semantics):** the harness (Go, `go-containerregistry`) walks the closure, fetches leaves-first with on-the-wire hashing, and publishes the root last. `docker kill peerA` mid-blob ⇒ the root never appears in `local` (I3); re-run resumes against `peerC`, skipping verified blobs (I4); `crane validate` green.
- 4. **Corruption:** flip one byte of a blob on the peer ⇒ digest mismatch, transfer aborts, nothing published, peer demoted (T1).
- 5. **Concurrency:** two simultaneous requests for the same digest coalesce to one transfer (I5).

The harness is deliberately shaped to become the seed of the FR6 multi-Satellite air-gapped e2e suite (extending `test/e2e/`, B11), so the application-time artifact converts directly into term deliverable work.

Failure-Mode Matrix

#	Failure	Detection	Behavior	Observable signal
1	Peer down before transfer	Availability HEAD timeout	Next peer; breaker increments	<code>peer_attempts_total{outcome="unreachable"}</code>
2	Peer dies mid-blob	Stream error / size short	Retry per policy → next peer; root unpublished	<code>fallback_total{reason}</code> + audit
3	Corrupt bytes	11 hash mismatch	Abort artifact from this peer; demote immediately ; try next source	<code>outcome="verify_failed"</code> + audit event
4	Slow peer	Phase timeouts	Cancel, backoff, breaker	duration histogram outliers
5	Divergent peers (different subsets)	Per-digest availability	Plan may split across peers per blob; correctness unaffected (digest-addressed)	per-peer fetch counters
6	Digest nowhere (no peer, no upstream)	Exhausted chain	Explicit failure, reason <code>not_found_anywhere</code>	<code>cache_fill_total{result="failed"}</code>
7	Local disk full mid-transfer	Write error	Abort; staged content GC-able; root unpublished	error log + audit
8	Satellite crash mid-import	Process restart	Re-plan; verified digests skipped (14); completes or fails cleanly	resume event in logs
9	Concurrent same-digest demands	Singleflight key	One transfer; others await result	dedup counter
10	Partial closure on peer (index present, child missing)	PLAN-phase HEADs	Peer skipped for this artifact (or per-blob split, #5); never partial-publish	<code>closure_incomplete_on_peers</code>

Performance & Resource Envelope

- **Throughput** is LAN-bound; the CPU cost is dominated by SHA-256 on the wire (hundreds of MB/s per core on typical edge x86/ARM — to be measured in the experiment transcript).

- **Memory** is $O(\text{blobs_per_transfer} \times \text{stream-buffer})$: streaming I/O end-to-end, no whole-blob buffering (NFR4).
- **Disk** during transfer is bounded by artifact size + staged-but-unpublished content, reclaimable by the janitor (I4).
- **Availability checking** costs one HEAD per unresolved digest per candidate peer, LAN-local; the batched check (OQ4) is the escape hatch if fleets make this measurable.
- **When disabled**, the feature contributes zero goroutines, zero sockets, zero config surprise (FR1).

Timeline

Twelve weeks (program window Sep 7 – Nov 27, 2026), each phase gated by an acceptance criterion:

Weeks	Deliverable	Acceptance
1–2	ADR-0010 (PR #638) iterated with maintainer review; scope + digest-domain rule settled	ADR-0010 approved (G1)
3–4	Config model + validation + peer client (mTLS, HEAD/GET, timeouts)	Unit tests green; misconfig $\Rightarrow$ fail-closed
5–7	Transfer engine: closure walker, streaming verify, root-last publish, resume, singleflight	Fault-injection unit suite green (mid-term demo) — kill/corrupt/restart scenarios from §10
8–9	Selection/health/breaker + fallback chain; metrics/logs/audit events	Failure matrix rows 1–6 demonstrably observable
10–11	Multi-Satellite air-gapped e2e in <code>test/e2e</code> (compose topology, no GC container); docs draft	e2e green including failure recovery (FR6)
12	Hardening, docs (FR8), buffer for review latency	Feature merged or in final review; retro

Dependencies

- `go-containerregistry` (already vendored) for v1 graph/verify primitives; `oras-go/v2` alignment as ADR-0009 lands (B5/B6).
- Embedded Zot behavior only as *today's* local store behind the store seam — no Zot-specific API dependence (B5, NFR3).
- Existing TLS/identity surface (B8) and audit pipeline (B10).
- `test/e2e` harness (B11) for the FR6 environment.
- **Not** dependent on: Ground Control availability, Internet egress, Kubernetes.

Alternatives Considered

Alternative	Why not
Custom P2P transfer protocol	Reinvents Distribution semantics; loses BYO portability; enlarges the security surface for zero capability gain — peers already speak Distribution (B7)
Zot sync extension as the mechanism	Couples to a component leaving the target architecture (B5); tag/period-oriented; build-tag availability caveats (Gap Analysis)
Node-level cache (Spegel-style)	Ephemeral, cluster-internal semantics; Satellite requires durable, auditable registry import that survives node churn and works without k8s
DHT/IPFS content network	Open-world discovery contradicts allowlisted air-gap trust; operationally heavy on edge hardware
Do nothing (status quo)	U1–U4 costs persist; the listing's air-gap requirement is unmet
Publish-then-verify (optimistic import)	Violates NFR5/I3; the experiment's violation demo (§10.2) is exactly this failure made visible

Operations

- **Enablement:** per-Satellite config flag, default off; canary one peer pair, then widen. Disable is instant and safe (in-flight transfers abort; nothing partially visible; staged content GC'd).
- **Downgrade:** no schema/state migrations; removing config restores prior behavior bit-for-bit.
- **Troubleshooting:** every failure row in §11 maps to a metric + log event; the doc set (FR8) includes a symptom→signal table and a `satellite peers status`-style diagnostic (CLI surface to be aligned with existing `cmd/` patterns).
- **Fleet hygiene:** peers in `revoked` lifecycle are excluded automatically (B9); operators rotate peer TLS material through the same channels as existing Satellite TLS (B8).

Security / Privacy / Compliance

- **Supply-chain integrity:** unchanged trust root — content identity is the OCI digest; peers add availability, not authority (§9.6). Signature/attestation graphs can travel with content via the referrers expansion without new verification semantics (N4).
- **Transport:** mTLS peer↔peer; identity pinned to configured peer entries; SPIFFE-ready (B8/ADR-0005).
- **Auditability:** every import records source peer, digest, outcome through the existing audit sinks (B10) — the compliance story for air-gapped sites is "every byte's provenance is logged and hash-verified."
- **Privacy:** no new data classes; peer traffic stays on the local segment; no telemetry leaves the site that doesn't already.
- **Egress posture:** the feature *reduces* WAN egress and introduces none of its own.

Risks

Risk	Mitigation
ADR-0009 churn lands mid-term and moves the store/copy substrate	The source abstraction (§9.1) is the seam; engine logic (closure/verify/publish) is substrate-independent; deliverables sequenced so the ADR settles interfaces first
Review bandwidth on a fast-moving repo	ADR-first process; small PRs mapped to §13 phases; e2e evidence attached to each
Zot behavioral variance / removal timing	Distribution-primitives-only baseline (NFR3); store behind a seam
Peer misconfiguration at fleet scale	Fail-closed validation (§9.7); explicit reasons; docs symptom table
Scope creep toward discovery/federation	Non-goals N1/N3 enforced; future-work parking lot (§19)
Digest-domain interactions with legacy imports (B3)	Strict rule + detection metric (§9.4); surfaced as OQ2 rather than silently decided

Future Work

- **Batched availability API** (OQ4) once HEAD fan-out is measurable.
- **Peer discovery** (mDNS/DNS-SD; SPIFFE-federated attestation) layered on the same source abstraction.
- **Referrers-aware default policies** (e.g., "always carry signatures for repos matching X").
- **Cross-site federation** over intermittent WAN with scheduling/backpressure.
- **Bandwidth-aware plan splitting** across multiple peers per closure (matrix row 5 formalized into an optimizer).
- **Policy-differentiated peers** (per-peer artifact-type or namespace policies) under the ADR-0009 policy engine.

Open Questions for Maintainers

#	Question
OQ1	Staging mechanism preference: hold-root-last into the live store (proposed) vs. a staging repository namespace vs. relying on upload-session semantics — any operational constraints from existing deployments?
OQ2	Digest-domain policy (§9.4): should the legacy media-type normalization (B3) be retired fleet-wide as ADR-0009 lands, and should desired state carry source-domain digests only?
OQ3	Peer serving in BYO mode: is "bare registry as peer" (no Satellite process on the serving side) a supported topology for v1, or must both ends run Satellite?
OQ4	Appetite for a batched availability endpoint , and if so, whether it should live under a Satellite-owned route rather than <code>/v2/</code> ?
OQ5	Metric naming conventions to match existing/planned Satellite telemetry?
OQ6	Policy gate for peer-sourced content: confirmed that peer fills traverse the same ADR-0009 evidence/policy stages as upstream fills (this RFC assumes yes)?

Audit Log

Date	Version	Change
2026-08-18	v0.1	Initial draft for discussion
2026-08-18	v0.2	Author attribution (Harshita Kumari); integrated <code>oci-closure-verify</code> harness + demo, PR #637 (digest-drift fix), PR #638 (ADR-0010 draft)

References

- Upstream: harbor-satellite#542 · Program: cncf/mentoring#1999
- ADRs: 0009 — Transparent OCI registry proxy · 0005 — SPIFFE identity · 0006 — Lifecycle states · 0001/0002 (historical)
- Code anchors: `internal/satellite/state/replicator.go` , `pkg/config/config.go` , `internal/satellite/registry/` , `test/e2e/` , `docker-compose.byo.yml` , `go.mod`
- Specs: OCI Distribution Spec, OCI Image Spec (descriptor graphs, referrers)
- Prior art: Dragonfly, Kraken, Spegel, Zot sync
- Author artifacts: `oci-closure-verify` verification harness (demo) · PR #637 digest-drift fix + regression test · PR #638 ADR-0010 draft

- Community findings credited in §7: [1amKhush](#) (tag/digest divergence reproduction), [shreyashsri79](#) (ADR-0009 transition & build-tag archaeology), [harshal0704](#) (BYO portability; state semantics)